
TIBER CAD

Examples Book *Release 2.0*

TiberLAB srl

April 1, 2011

CONTENTS

I	Examples	1
1	Elasticity	3
1.1	Piezoelectric nanogenerator	3
2	Drift Diffusion	7
2.1	Tutorial Bulk Si	7
2.2	Tutorial Si-Pn Diode	13
2.3	n-MosFET 2D	23
2.4	Bjt1 & Bjt2	31
3	Thermal	39
3.1	Boundary conditions	39
3.2	Heat sources	40
4	Envelope Function Approximation	43
4.1	Quantized states in a GaN/AlGaIn quantum well	43
5	Simulation Dye Solar Cells	53
5.1	Device structure	54
II	Glossary	61
	Bibliography	117
	Index	119

Part I

Examples

ELASTICITY

1.1 Piezoelectric nanogenerator

In this example we will show how to perform electro-mechanical simulation of a piezoelectric nanogenerators [Wang] . The simulation domain consists of a Zinc oxide nanowire (length = 200 nm, diameter = 50 nm) growth on a ZnO substrate and surrounded by air. The performance of such a device can be obtained by computed the piezopotential produced when the nanowire is subjected to a shear force.

With the block device we specify the materials and properties associated with the physical regions. In particular, we have the *Nanowire*, *Substrate* and *Air* .

Device

```
{
  dimension = 3
  meshfile = mesh.msh
  mesh_units = 1e-9
  material = AlGaIn
  x = 0.2
  x-growth-direction = (-1,0,1,0)
  y-growth-direction = (-1,2,-1,0)
  z-growth-direction = (0,0,0,1)
  Region Nanowire
  {
    doping_type = donors
    doping_level = 0.035
  }
  Region Substrate
  {
    doping_type = donors
    doping_level = 0.035
  }
  Region Air
  {
```

```
        material = Air
    }
}
```

In this example we use a isotropic stiffness constant and, therefore, a Young modulus and Poisson's ration must be defined. The base of the substrate is clamped whereas a shear force is applied on the top of the nanowire.

```
Module elasticity
{
    Physics
    {
        Stiffness isotropic
        {
            Young = 129 Poisson = 0.349
        }
        Contact Base
        {
            type = clamp
        }
        Contact Top
        {
            type = surface_force force = (0.0, 0.0625,0.0)
        }
    }
}
```

The driftdiffusion module computes the piezopotential due to the applied force. The piezopolarization uses the strain computed by the elasticity module. The base of the substrate is grounded.

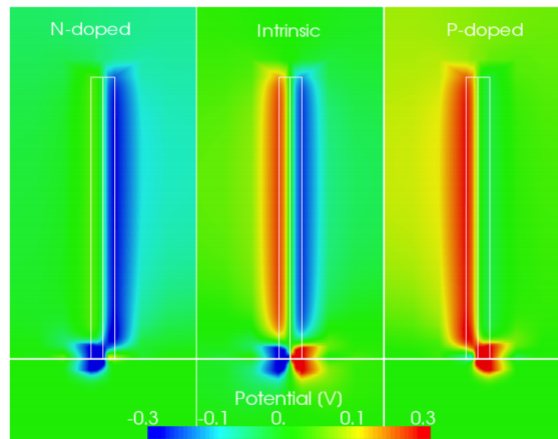
```
Module driftdiffusion
{
    Physics
    {
        {
            strain_simulation = elasticity
        }
        Contact Base
        {
            Voltage = 0.0
        }
        Polarization piezo
        {
        }
    }
}
```

In the simulations block we specify the simulations to be solved. The default simulation name is the name of the module itself.

Simulations

```
{  
  solve (elasticity,driftdiffusion)  
}
```

Consistently with results published in [Gao] the free carriers tend to screen the output potential. The potential profile is shown below.



DRIFT DIFFUSION

2.1 Tutorial Bulk Si

In this example we will see a very simple TiberCAD simulation:

1D calculation of Poisson and drift-diffusion for a bulk Silicon sample.

The following files should be in your working directory:

`bulk.tib` : **TiberCAD** input file

`bulk.msh` : mesh file

The mesh file can be obtained from the following GMSH geo file : `bulk.geo`

This is the summary of the Sections of this Tutorial :

- *Step 1 - Modeling the device*
- *Step 2 - Meshing the device*
- *Step 3 - TiberCAD Input file*
- *Step 4 - Run TiberCAD*
- *Output*

2.1.1 Step 1 - Modeling the device

As a first step, we have to model the device. To do so, you can use DEVISE module of ISE-TCAD 9.5 software package or GMSH program.

Here we'll see in details the procedure for GMSH.

There are two possible ways to use GMSH:

Interactive, using the graphical interface Using a script file In the following we'll see how to write a basic GMSH script (bulk.geo); for any details please refer to GMSH manual GMSH (<http://geuz.org/gmsh/>).



: In a **1D** simulation it is assumed that the geometrical model is restricted to the **x axis** . In a **2D** simulation it is assumed that the geometrical model is restricted to the **xy-plane (z=0)** Any other geometrical orientation could give unpredictable results

In a GMSH script, several variables can be defined and given a value in this way:

```
L = 1  
d = 0.01
```

these are valid GMSH variables: L is just the length of the Si sample; d is the value of a *characteristic mesh length* (see below).

Definition of geometrical entities **Points**

```
Point(1) = {0, 0, 0, d}  
Point(2) = {L, 0, 0, d}
```

The first three expressions inside the braces on the right hand side give the three X, Y and Z **coordinates** of the point; the last expression (**d**) sets the **characteristic mesh length** at that point, that is the “size” of a mesh element, defined as the length of the segment for a line segment, the radius of the circumscribed circle for a triangle and the radius of the circumscribed sphere for a tetrahedron.

Thus, the smaller is the value of d, the greater is the mesh density close to that point.

The size of the mesh elements will then be computed in GMSH by linearly interpolating these characteristic lengths in the whole mesh.

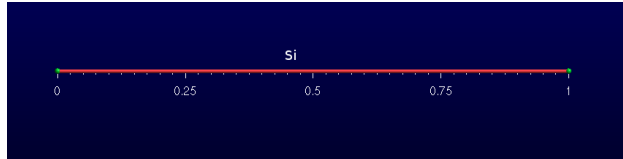
Definition of a geometrical entity **Line**

```
Line(1) = {1, 2}
```

The two expressions inside the braces on the right hand side give the identification numbers of the start and end points of the line.

Definition of the physical entity **Physical Line bulk**

```
Physical Line("bulk") = {1}
```



The expression(s) inside the braces on the right hand side give the identification numbers of all the **geometrical lines** that need to be gro ed inside the *physical line* .

In this way, in general, physical regions are created which associate together geometrical regions, and then the related mesh elements, which share some common physical properties. It's only these physical regions which can be referred to outside GMSH. In TiberCAD, this is done by associating one or more physical regions to a **TiberCAD region** through the keyword *mesh_regions* (see in the following).

Definition of two physical entities Physical Point:

```
Physical Point("anode2")    = {1}
Physical Point("cathode")   = {2}
```

|warn| : In general, in a nD simulation, ***(n-1)D*** physical regions (points in 1D, are used by TiberCAD to impose the required boundary conditions.

Each (n-1)D physical region defined in this way in GMSH will be associated in TiberCAD to a boundary condition region, through the keyword **BC_reg_numb** . Thus, in this case, Physical points Anode and Cathode will be associated respectively to two **Contact** regions (see in the following).

2.1.2 Step 2 - Meshing the device

The *.geo* script file with the geometrical description can be run in GMSH, to display the modelled device and to mesh it through the GMSH graphical interface. Alternatively, a *non-interactive* mode is also available in GMSH, without graphical user interface.

For example, to mesh this 1D tutorial in non-interactive mode, just type:



```
gmsht bulk.geo -1 -o bulk.msh
```

where *bulk.geo* is the geometrical description of the device with GMSH syntax:

-1 means *1D mesh generation*

some command line options are:

-1 , -2, -3 to perform *1D, 2D or 3D* mesh generation,

-o **mesh_file.msh** to specify the name of the mesh file to be generated

In this way, a .msh has been generated and is ready to be read in TiberCAD.

2.1.3 Step 3 - TiberCAD Input file

Now we have to write down the **TiberCAD input file** ([bulk.tib](#)). For a detailed reference see the user guide (Input file) and Getting started 1

2.1.4 Description of Device Regions

First, we have to list all the **TiberCAD Regions** present in our Device: a TiberCAD Region is usually a section of the device featuring the same material and possibly the same doping.

```
Device
{
    meshfile = bulk.msh

    Region bulk
    {
        material = Si

        Doping
        {
            Nd = 1e16
            type = donor
        }
    }
}
```

The TiberCAD Region bulk is made of Silicon and n-doped with a concentration $1 \times 10^{16} \text{cm}^{-3}$.

Through the keyword **Region** , one GMSH physical region (Physical Lines in 1D, Physical Surfaces in 2D, Physical Volumes in 3D) previously defined in the GMSH mesh ([Step 1 - Modeling the device](#)), can be associated to the present TiberCAD Region, in this way:

```
Region GMSH_physical_region_name
```

In this case, the Physical Line bulk is associated to the TiberCAD Region bulk.

Alternatively, through the optional keyword **mesh_regions** , one or more GMSH physical regions can be associated to a single TiberCAD Region .

2.1.5 Definition of Simulation

Now we define the **Simulation** driftdiffusion_1: it belongs to the class **driftdiffusion**

```

Module driftdiffusion
{
    name = driftdiffusion # this is the default name

    regions = all          # 'all' is the default

    # what we want to plot
    plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)
    ....

```

The TiberCAD simulation *driftdiffusion_1* , belonging to the model driftdiffusion, will be applied to the whole device structure (`regions = all`)

2.1.6 Definition of Boundary Conditions

The anode and cathode contacts of our 1D Si sample are defined as **Boundary conditions regions** (`Contact anode`, `Contact cathode`) in the following way:

```

Module driftdiffusion {
    name = driftdiffusion
    regions = all
    plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)
    Contact anode { voltage = $Vb } Contact cathode { }
}

```

Through the keyword `Contact` , one (n-1) -dimension GMSH physical region (Physical Point in 1D, Physical Line in 2D, Physical Surface in 3D) previously defined in the GMSH mesh ([Step 1 - Modeling the device](#)), can be associated to the present TiberCAD Contact, in this way:

`Contact GMSH_physical_region_name`

In this case, the *Physical Point* **anode** is associated to the TiberCAD Contact anode and the Physical Point cathode is associated to the TiberCAD Contact cathode.

Both contacts are defined as *ohmic* , cathod is assigned a fixed `voltage = 0.0` , while anode voltage is given by the value of the variable *Vb*

```
voltage = @Vb
```

2.1.7 Definition of Simulation parameters

Vb is specified in the sweep block, in the Solver section

```
Module sweep
{
  solve = driftdiffusion
  variable = $Vb
  start = 0.0
  stop = 1
  steps = 10
  plot_data = true
}
```

In this way, the simulation `driftdiffusion_1` is performed for 10 (`steps = 10`) values of the anode voltage (`variable = Vb`), between 0 and 1.

For each step we want to plot the solution variables specified in the `driftdiffusion` module (`plot_data = true`).

2.1.8 Definition of Execution parameters

In the **Simulation** section , we decide *which* simulations to perform and in which *order*; set `solve = sweep` , to execute the sweep which run `driftdiffusion_1` simulation for the specified loop.

```
Simulation
{
  verbose = 2

  solve = sweep

  resultpath = output
  output_format = grace
}
```

Output files with conduction and valence band profiles (`plot = Ec,Ev..`) and all the calculated values of the current at the contacts (*ContactCurrents*) (the IV characteristic) are generated.

To increase the amount of information written to the screen we can vary the verbose level (`verbose = 2`).

2.1.9 Step 4 - Run TiberCAD

Now we can run TiberCAD

by double clicking on `bulk.tib` file (in Windows)

or by command line in linux: `tibercad bulk.tib`

2.1.10 Output

The generated Output files are:

- `driftdiffusion_materials.dat` : material (mesh) regions, in this case just region 1
- `driftdiffusion_nodal.dat` : nodal quantities (here conduction and valence band)
- `sweep_driftdiffusion_Vb.dat` : integrated current at the two contacts for each sweep step

Attachment Size

`bulk.tib` 1.16 KB

`bulk.geo` 181 bytes

`bulk.msh` 4.49 KB

2.2 Tutorial Si-Pn Diode

In this first tutorial we will show a simple 1D Si pn diode example. We will calculate the IV characteristic of the pn diode, by solving the Poisson equation and calculating the current with a drift-diffusion scheme.

The semi-classical transport simulation of electrons and holes is based on the driftdiffusion approximation .

Beside the electric potential, the electro-chemical potentials are used as variables such that the system of PDEs to be solved reads as follows:

$$\begin{aligned}
 -\nabla(\varepsilon\nabla\varphi - \mathbf{P}) &= -e(n - p - N_d^+ + N_a^-) \\
 -\nabla(\mu_n n \nabla\phi_n) &= R \\
 -\nabla(\mu_p p \nabla\phi_p) &= -R
 \end{aligned}
 \tag{2.1}$$

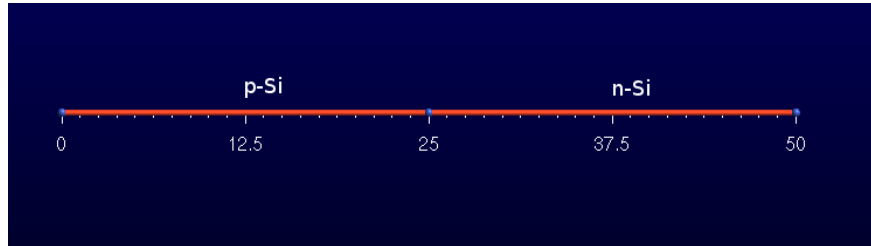
where $\mathbf{P}$ is the electric polarization due to e.g. piezoelectric effects and $\mathbf{R}$ is the net recombination rate, i.e. recombination rate minus generation rate.

In order to execute correctly the example you should have the following files in the same working directory:

`diode_1D.tib` : input file for TiberCAD `diode_1D.msh` : mesh file produced by GMSH (<http://geuz.org/gmsh/>).

GMSH is an automatic three-dimensional finite element mesh generator. Please refer to GMSH manual for any details.

The mesh has been obtained from the GMSH geo script file `diode_1D.geo` .



2.2.1 Modeling of the device with GMSH

In this script, first the geometrical entities Points and Lines are defined (see *Tutorial Bulk Si*). Then, two physical regions are defined, each associated to one of the two geometrical entities: Physical Line("p_side") and Physical Line("n_side").

```
Physical Line("p_side") = {1}  
Physical Line("n _side")= {2}
```

In this way, these physical regions are made available for **TiberCAD** , and will be associated to a **TiberCAD Region** (see in the following).

Then, two **Physical Points** are defined, **anode** and **cathode** , and associated to the first and to the last point of our 1D device.

```
Physical Point("anode") = {1}  
Physical Point("cathode") = {3}
```

These points are needed to impose some boundary conditions and in this way they are made available for **TiberCAD** , and will be associated each to a boundary condition region (see in the following).

A *non-interactive* mode is also available in GMSH, without graphical user interface. For example, to mesh this first 1D tutorial in non-interactive mode, just type:



```
gmsh diode.geo -1 -o diode.msh
```

where `diode.geo` is the geometrical description of the device with GMSH syntax;

some command line options are:

- -1 , -2, -3 to perform 1D, 2D or 3D mesh generation

- -o **mesh_file.msh** to specify the name of the mesh file to be generated

2.2.2 TiberCAD input file

This is the summary of the Sections of this Tutorial :

- *Device structure*
- *Simulation models*
- *Solver parameters*
- *Physics parameters*
- *Run simulations*
- *Output*

2.2.3 Device structure

Let's have a look to the input file; for further details you can refer to the reference manual.

```
Device
{
  meshfile = diode_1D.msh
  material = Si

  Region n_side
  {
    Doping
    {
      Nd = 1e18
      type = donor
    }
  }

  Region p_side
  {
    Doping
    {
      Nd = 1e18
      type = acceptor
    }
  }
}
```

First we define the device structure: it is composed by two Regions, both constituted of the material Silicon, respectively n and p doped with a concentration of $1e18\text{ cm}^{-3}$.

With **Region n_side** and **Region p_side**

the physical regions previously defined in GMSH are associated to the respective **TiberCAD Region** .

2.2.4 Simulation models

Now, we have to define the **simulations** which we are going to execute in our calculations.

One **simulation** is a particular set of equations, boundary conditions, physical parameters, solver parameters, which describes a physical problem to be solved by TiberCAD.

A valid **TiberCAD simulation** must belong to one of the predefined **TiberCAD simulation models** .

TiberCAD, being a **multiphysics** software tool, includes several simulation models, each one describing a physical problem to be solved, e.g DriftDiffusion (to solve Poisson and DriftDiffusion equations) , EFASchroedinger (to solve Schroedinger equation in effective mass approximation) , Macrostrain (to calculate macroscopical strain) and others.

To create a **TiberCAD simulation** , we first have to declare the **TiberCAD model** class to which our simulation belongs:

```
Module driftdiffusion
{
    name = dd

    plot = (Ec, Ev, Eg, ElField, ElPotential, eQFermi, hQFermi, eDensity,
           hDensity, NetRecombination, eCurrentDensity, hCurrentDensity,
           CurrentDensity, ContactCurrent, eMobility, hMobility,
           IonizedDonors, IonizedAcceptors)
```

Here, we declare that the simulation to be created will belong to the model class **driftdiffusion** (`model driftdiffusion`)

In the next Options block, we define the name of the TiberCAD simulation (`name = dd`) and the TiberCAD regions to which it will be applied (`region = Regiona_Name`) if you don't specify anything, it means the whole device).

After we specify types of data we want in our Output file.

In this example, just one simulation for the *driftdiffusion model* is created.

In general, several TiberCAD simulations belonging to the same **model** can be created; each of them must have a different **name** . This name must also be different from the **model name**.

As we will see in the following, it is possible to specify physical and solver parameters for a single **TiberCAD simulation** , by referring to its **name** .

Anyway, it is also possible to specify parameters common to all the **TiberCAD simulations** belonging to the same **model** , by referring to the name of the **TiberCAD model** instead (in this example driftdiffusion):

```
Physics
{
  particle_density
  {
    statistics = boltzmann
  }

  # use a field dependent mobility model with the
  # gradient of the electrochem. potential as field
  # parameter
  mobility field_dependent
  {
    low_field_model = doping_dependent
  }
  # use SRH and Auger recombination
  recombination (srh, auger) {}
}

Contact anode
{
  type = ohmic
  voltage = $Vb[0.0]
}

Contact cathode
{
  type = ohmic
  voltage = 0.0
}
}
```

Thus, in this simulation we are going to calculate *Poisson* and *Drift-diffusion* (**model driftdiffusion**) for all the device (**regions = all** if not specified).

Next, some `physical_model` blocks follow. The `physical_model` blocks are used in general to define the physical model to be used to describe a physical property or quantity associated to the present TiberCAD model.

Here, for example, we choose a Schottky-Read-Hall model (`model = srh`) to describe recombination in our driftdiffusion calculations.

```
physical_model recombination
```

```
    Physics
```

```
{ recombination (srh, auger) {}  
}
```

Also, we choose a field dependent model to describe electron and hole mobility (`mobility = field_dependent`), with the low field mobility determined by a doping-dependent model (Masetti model, see reference manual) (`low_field_model = doping_dependent`).

```
physical_model electron_mobility { model = field_dependent low_field_model = dop-  
ing_dependent }
```

Now, we have to specify the Boundary conditions for the PDE problem represented by the simulation `dd` : they are defined in `Contact` section.

Here, two boundary conditions are defined, corresponding to the two contacts of our pn diode, anode and cathode.

```
Contact anode  
{  
  type = ohmic  
  voltage = $Vb[0.0]  
}
```

```
Contact cathode  
{  
  type = ohmic  
  voltage = 0.0  
}
```

Here the **anode** and **cathode** Contacts are associated to the corresponding anode and cathode regions defined in GMSH as Boundary condition region `s` (which in the present case of 1D simulation are actually just a point), with

- *Contact anode*

and

- *Contact cathode* .

Besides, the type of Boundary condition is assigned (in this case **ohmic**) with its (initial) value, given by **voltage** .



The anode voltage is expressed by the notation **@Vb[0.0]**. This means that the anode voltage will be given the value of the variable **Vb** , specified in the **sweep** block (see after). **[0.0]** means that the default voltage value is 0.0 V.

2.2.5 Solver parameters

Definition of Model-dependent Solver parameters

```
Module sweep {  
    variable = $Vb solve = dd  
    start = 0.0 stop = 1.2 steps = 60  
    plot_data = true # to write data for each step  
}
```

Now, the parameters for the **solver** are defined.

These values are tuned for the specific simulation and usually don't need to be changed; however you can refer to the manual for further details.

The **sweep** block allows us to perform the calculation of the IV characteristic of the diode: it defines the variable *Vb* , which will assume a range of values between 0 and 1.2 V. These values will be assigned in turn to the *anode* contact, in order to execute a series of complete dd simulations (simulation = dd) for each bias condition.

2.2.6 Physics parameters

Definition of Model dependent physical parameters:

```
Physics  
{  
    particle_density  
    {  
        statistics = boltzmann  
    }  
    mobility field_dependent  
    {  
        low_field_model = doping_dependent  
    }  
    recombination (srh, auger) {}  
}
```

In the Physics section usually some physical parameters are defined. In this case we state that the Fermi-Dirac statistic will be applied (statistics = FD).

2.2.7 Run simulations

Finally, in Simulation section, we define the actual simulation to be performed, specified by:

```
solve = sweep
```

this determines the execution of dd inside the *sweep* cycle; this means that we are going to perform a calculation of **Poisson** and **drift-diffusion** for all the biases defined in the sweep section.

Results of the calculation will be in the format of xmgr data visualization program (`output_format = grace`), suited to 1D case, since it consists of ascii text files with column data.

```
Simulation
{
    verbose = 2

    solve = sweep

    resultpath = diode1D_results
    output_format = grace
}
```

Note that the output variable `ContactCurrents` is needed to get in output the IV characteristic.

Now we can run TiberCAD....

```
tibercad diode.tib
```

During the execution, the screen output shows some information about how the simulation is going:

First, about the creation of the Device regions; if there is an inconsistency between the mesh file and input file device description, here an error message is displayed and the execution terminated.

If all is ok, the calculation is started:

```
We solve: sweep
```

In this example only one simulation is performed, sweep, which is a set of drift-diffusion calculations.

First, the equilibrium solution is found, by solving Poisson equation:

```
DriftDiffusion (name: driftdiffusion) Solving
equilibrium ..... iterations: 6, residual =
1.95054e-16 Equilibrium done
```

Then, current continuity equations are solved together with Poisson eq., for each step of the sweep.

The voltage and current found for each contact is displayed:

```
Sweep value Vb = 0.05
```

```
<<-----
```

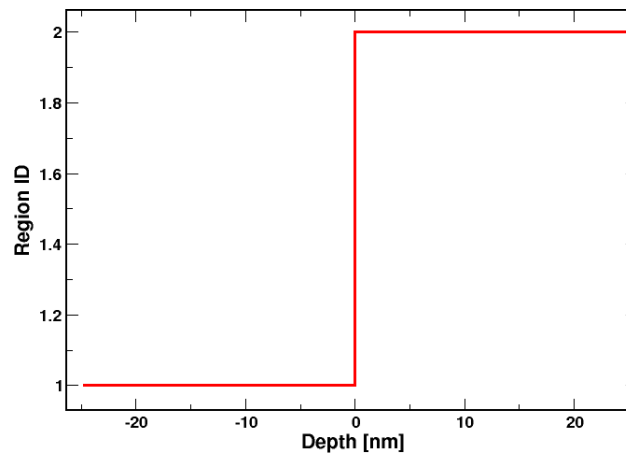
```
iterations: 5, residual = 1.17131e-10
```


contact name	contact voltage	contact current
cathode	0	6.31825e-05
anode	0.05	-6.3187e-05

The simulation ends correctly (hopefully...) with the Sweep value $V_b = 1.2$.

2.2.8 Output

Now, the output directory contain the simulation results, as defined in `output_format` and **plot**.



Thus, in `driftdiffusion_materials.dat` we have the information about the material regions of the device (not very interesting in 1D, but useful in 2 and 3D).

In `driftdiffusion_nodal_Vb_0.000.dat` and all the other files for each bias step, we have the output for the nodal quantities which have been calculated, e.g. conduction and valence bands, (quasi)fermi levels, electron and hole density and mobility.

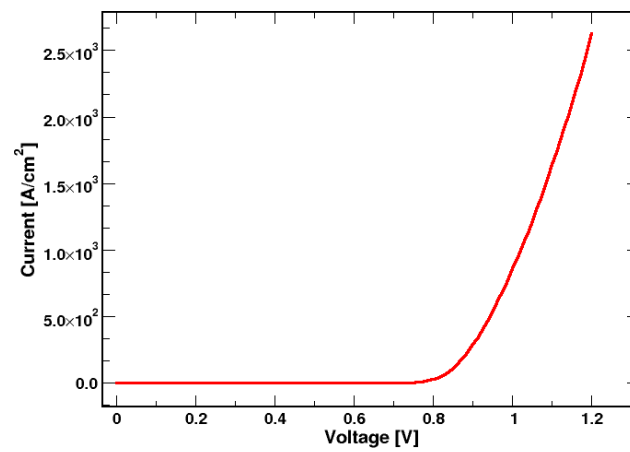
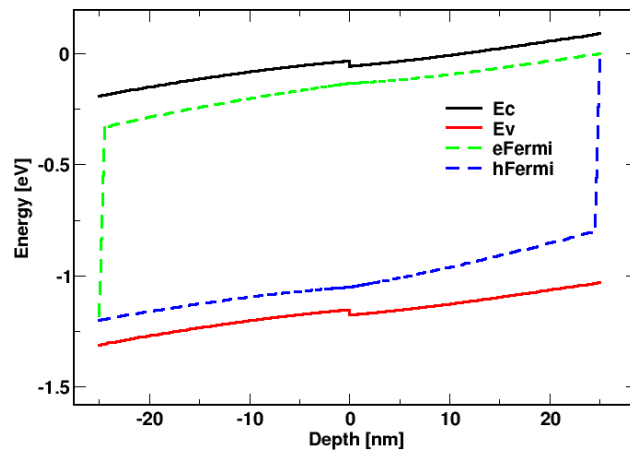
Here is, for example, the band profile obtained at the last step of calculation, for $V = 1.2V$.

The IV characteristic of the diode is contained in the file `sweep_driftdiffusion_Vb.dat`, where the currents at both the contacts are reported.

In the file `driftdiffusion_elemental_Vb_0.200.dat` etc., you can find the elemental quantities which have been calculated, in this case the electron, hole and total current density (J_{n_x} , J_{p_x} , J_x).

Attachment Size

[diode_1D.tib](#) 1.79 KB



diode_1D.geo 239 bytes
diode_1D.msh 26.24 KB

2.3 n-MosFET 2D

This is the summary of the Sections of this Tutorial :

- *Device structure*
- *Drift-diffusion simulation*
- *Calculation of IV drain characteristic*
- *Physic*
- *Output*
- *Calculation of transfer characteristic*

In this tutorial we will show an example of **2D** simulation of a **Si n-MOSFET** device.

We will calculate :

- IV drain characteristic,
- Id/Vg transfer characteristic.

In order to execute correctly the example you should have the following files in the working directory:

`mosfet.tib` : input file for TiberCAD

`mosfet.msh` : mesh file produced by GMSH from the script `mosfet.geo`

Here is the geometry of the Mosfet device as it is meshed by GMSH.

Let's give now a look to the input file; for further details you can refer to the program reference manual.

2.3.1 Device structure

Description of the device physical regions:

```
Device mosfet
{
  meshfile = mosfet.msh
  material = Si
```

```
Region substrate
{
  Doping
  {
    Nd = 1e18
    type = acceptor
  }
}

Region contact
{
  Doping
  {
    Nd = 5e19
    type = donor
  }
}

Region oxide
{
  material = SiO2
}
}
```

First we define the device structure: it is composed by three **TiberCAD Regions**: **substrate** and **contact** , both made of *Silicon* , and *oxide* (SiO2).

The **substrate** region (which include channel) is doped p-type $10^{18}cm^{-3}$; contact region is composed by the two regions around source and drain, both heavily doped n-type $5 \times 10^{19}cm^{-3}$; finally, **oxide** is the gate dielectric.

2.3.2 Drift-diffusion simulation

Now, we define the **TiberCAD simulation models** which will be used in this example. We are going to calculate Poisson and Drift-diffusion (model *driftdiffusion*) for all the device (regions = all in options).

A field dependent model for **mobility** is applied.

```
recombination srh {}

mobility
{
  type = field_dependent
  low_field_model = doping_dependent
}
```

Then, we have to specify the the three **contacts** of our Mosfet.

We assume a gate width of 1 mm.

Contact gate

```
{
  type = schottky
  barrier_height = 3.0
  voltage = $Vg
  area_factor = 0.1
}
```

Contact source

```
{
  voltage = 0.0
  area_factor = 0.1
}
```

Contact backcontact

```
{
  voltage = 0.0
  area_factor = 0.1
}
```

Contact drain

```
{
  voltage = $Vd
  area_factor = 0.1
}
```

The contacts (boundary regions for this model) have to be associated each to the corresponding region defined in the mesher program (here GMSH) as a Boundary region (which in the present case of 2D simulation is a line (`Physical Line` in GMSH)).

This is made simply with

Contact gate

and so on for the other contacts.

The **gate** contact is defined as a **schottky** contact, with a barrier value depending on the gate metal workfunction (`barrier_height`).

The gate voltage is expressed by the notation `@Vg[0.0]` . This means that the gate voltage will be given the value of the variable *Vg* , specified in one of the sweep block (see after). **[0.0]** means that the default voltage value is 0.0 V. Source and drain contacts are defined as ohmic; while source is fixed to 0 (`voltage = 0.0`), drain voltage is expressed, as for the gate, by the value of the sweep variable *Vd* (`voltage = @Vd[0.5]`).

2.3.3 Calculation of IV drain characteristic

Definition of Model-dependent Solver parameters:

```
Module sweep
{
    name = sweep_drain
    solve = driftdiffusion

    variable = $Vd
    start = 0.0 # 0.2
    stop = 1.0
    steps = 5 # 4
}
```

```
Module sweep
{
    name = sweep_gate
    solve = driftdiffusion

    variable = $Vg
    start = -0.5
    stop = 1.5
    steps = 100

    max_step = 0.1
    plot_data = true
}
```

Every cycle is performed at maximum step of 0.1 V

Definition of model-independent parameters of the Simulation:

```
Simulation
{
    temperature = 300
    solve = (sweep_drain, sweep_gate)
    resultpath = output_transchar

    output_format = vtk
}
```

Now, the parameters for the solver are defined, for the driftdiffusion model. `coupling = electrons` specifies that only one type of carrier is considered.

`ksp_type = bcgsl` defines the solver type,

`nonlinear_solver = tiber` specifies the non-linear solver.

As a first step, we are going to calculate **Id/Vd drain current characteristic** .

We include the optional **Sweep** block, whose syntax is the following:

```
Module Sweep
{
  name = sweep_name1
  {
    .....
  }
}
Module Sweep
{
  name = sweep_name2
  {
    .....
  }
}
```

Here we define two nested **sweeps** : the external sweep (`sweep_gate`) calculates the drain current for a series of gate voltages, from -0.1 to 0.5 V; for each calculation, a sweep on drain voltages is performed (`simulation = sweep_drain`), defined by `sweep_drain` . In `sweep_drain`, `driftdiffusion` is calculated (`simulation = driftdiffusion`) for a series of drain voltages from 0 to 2 V.

2.3.4 Physic

In the **Physics** section usually some physical parameters are defined. In this case we just state that we want to use the **unstrained** model for `driftdiffusion` (since no strain calculation is intended) and that we use Fermi-Dirac statistic

```
Physics {
  particle_density {
    statistics = fermidirac
  }
  recombination srh {}
  mobility {
    type = field_dependent low_field_model = doping_dependent
  }
}
```

Finally, in **Simulation** section, we define the 2D (`dimension = 2`) simulation to be performed, specified by

```
solve = sweep_gate : this means that we are going to perform a calculation
```

of **Poisson** and **drift-diffusion** with a double sweep on gate and drain voltage, to calculate **Id/Vd drain current characteristics** .

Results of calculation will be in the format of **paraview** data visualization and post-processing program (`output_format = vtk`)

In **plot** we define the output variables to be calculated.

```
Simulation
{
    temperature = 300

    solve = sweep_gate
    resultpath = output_outputchar

    output_format = vtk
}
```

Now we can run TiberCAD....

```
tibercad mosfet.tib
```

2.3.5 Output

After the execution, the output directory contain the simulation results, as defined in `output_format` and `plot`.

TiberCAD s ports 3 packages for 2D and 3D data visualization and post-processing:

Free:

1. GMV, <http://www-xdiv.lanl.gov/XCM/gmv/GMVHome.html>

```
output_format = gmv
```

2. Paraview, <http://www.paraview.org/New/index.html>

```
output_format = vtk
```

Commercial:

1. Tecplot-ise, <http://www.amtec.com/>

```
output_format = ise
```

Let's see how to use Paraview to plot TiberCAD 2D results:

First, open the `.vtk` file from your directory:

The name of the loaded files will be shown in the Pipeline browser.

To visualize the content of the file, you should click on the `Apply` button in the **Properties** tab in **Object Inspector**

To select the output variable , go to **Display** and choose from the menu “Color by”, e.g. `electron_density`. Also, in Display section Scale and legend bar can be setted.

Thus, in `driftdiffusion_materials.vtk` we have the information about the material regions of the device: here, **1** indicates the substrate Region , **2** the contacts region and **3** the SiO₂ gate dielectric.

In `driftdiffusion_nodal.vtk` and all the other files for each bias step, we have the output for the nodal quantities which have been calculated, e.g. conduction and valence bands, (quasi)fermi levels, electron and hole density and mobility.

For example, this is the electron density for a drain voltage = 2V and a gate bias $V_g = 1V$. You can see the large increase of electron density in the channel, which is pinched off close to drain contact due to the high drain voltage.

In `sweep_2_driftdiffusion_Vg_0.000_Vd.dat` and all the other files for each V_g bias step (`sweep_2_driftdiffusion_Vg_step_Vd.dat`) we have the IV drain characteristics for each V_g bias.

Here are the IV characteristics obtained for a gate voltage V_g between -0.1 and 0,5 V.

2.3.6 Calculation of transfer characteristic

As a second step, we are going to calculate I_d/V_g transfer characteristic. (see input file `mosfet_transchar.tib`)

To do so, the two **sweeps** have to be defined in this way:

```
Module sweep
{
  name = sweep_drain
  solve = driftdiffusion

  variable = $Vd
  start = 0.0
  stop = 2.0
  steps = 40

  plot_data = true
}
```

```
Module sweep
{
  name = sweep_gate
  solve = sweep_drain
}
```

```
variable = $Vg
start = 0.0
stop = 1.5
steps = 6
}
```

the first sweep (sweep_drain) calculates the drain current (simulation = driftdiffusion) for a series of drain voltages, from 0 to 1 V, while Vg is kept at its initial value Vg=0; at the end of this calculation we have polarized the mosfet at a drain voltage of 1 V.

Then a sweep on gate voltages is performed, defined by sweep_gate . In sweep_gate , driftdiffusion is calculated (simulation = driftdiffusion) for a series of gate voltages from -1 to 0.5 V, by keeping the drain voltage at Vd = 1 V.

At the end, the Id/Vg transfer characteristic for Vd=1 V is obtained.

This time, in **Simulation** section, we define the 2D simulation to be performed by writing solve=(sweep_drain,sweep_gate) : this means that we are going to perform a calculation of **Poisson** and **drift-diffusion** with two sweeps in series, first a sweep on drain and than a sweep on gate voltage, to calculate **Id/Vg transfer characteristic** .

```
Simulation
{
  temperature = 300
  solve = (sweep_drain, sweep_gate)
  resultpath = output_transchar

  output_format = vtk
}
```

The rest of the input file remains unchanged.

Now we can run TiberCAD another time....

```
tibercad mosfet.tib
```

- Now, in the files sweep_1_driftdiffusion_Vd_step_Vg.dat (e.g. sweep_1_driftdiffusion_Vd_1.000_Vg.dat), we have the Id/Vg transfer characteristics for each Vd_step bias.

Here is the **Id/Vg transfer characteristic** for a drain voltage Vd = 1 V .

As before, in driftdiffusion_elemental.vtk and driftdiffusion_nodal.vtk , we have the output for respectively the nodal and the elemental quantities which have been calculated.

The **subthreshold S** parameter, given by

in this case results to be **~80 mV/dec** .

Attachment Size

mosfet.msh 309.63 KB

mosfet.geo 1.67 KB

mosfet.tib 2.28 KB

mosfet_transchar.tib 2.14 KB

2.4 Bjt1 & Bjt2

This is the example of the first Bjt.

2.4.1 Device

```
Device bjt
{
    material = Si
    meshfile = bjt.msh

    Region coll_doped # the buried doping
    {
        Doping
        {
            type = donor
            density = 1e19
        }
    }

    Region coll # the collector region
    {
        Doping
        {
            type = donor
            Nd = 5e15
        }
    }

    Region b_reg # the base region
    {
        Doping
```

```
        {
            type = acceptor
            density = 1e17
        }
    }

Region e_reg # the emitter region
{
    Doping
        {
            type = donor
            density = 1e19
        }
    }
}
```

2.4.2 Module

Module driftdiffusion

```
{

    name = dd

    # we use a different quadrature rule to avoid density peaks
    # where the mesh is not so dense
    quadrature_rule = trapez

    plot = (Ec, Ev, ElPotential, ElField, eQFermi, hQFermi, eDensity, hDensity,
            eCurrentDensity, hCurrentDensity, CurrentDensity, ContactCurrents,
            NetRecombination, eMobility, hMobility)

    Solver linesearch
    {
    }
}
```

Physic

Physics

```
{

    # use Fermi-Dirac statistics
    statistics = FD
}
```

```
mobility doping_dependent {}

recombination (srh, auger) {}
}
```

2.4.3 Contacts

Contact base

```
{
  type = ohmic
  voltage = $Vb
}
```

Contact collector

```
{
  type = ohmic
  voltage = $Vc
}
```

Contact emitter

```
{
  type = ohmic
  voltage = 0.0
}
```

2.4.4 Sweep

Module sweep

```
{
  name = sweep_base
  variable = $Vb
  start = 0.5
  stop = 0.7
  steps = 10

  # we calculate the output characteristics
  solve = sweep_coll

  # plot only for the last collector bias
  plot_data = true
}
```

Module sweep

```
{
  name = sweep_coll
  variable = $Vc
  start = 0.0
  stop = 3
  steps = 60

  solve = dd
  #plot_data = true
}
```

2.4.5 Simulation

The temperature = 300 is the default

```
Simulation
{
  verbose = 3

  solve = sweep_base

  resultpath = output
  output_format = vtk
}
```

2.4.6 Device

This is the example of the second bjt

```
Device bjt2
{
  material = Si
  meshfile = bjt2.msh

  Region coll_doped # the buried doping
  {
    Doping donor { density = 1e19 }
  }

  Region coll # the collector region
  {
    Doping
    {
```

```
        type = donor
        Nd = 5e15
    }
}

Region b_reg # the base region
{
    Doping
    {
        type = acceptor
        density = 1e17
    }
}

Region e_reg # the emitter region
{
    Doping
    {
        type = donor
        density = 1e19
    }
}

Region oxide
{
    material = SiO2
}
}
```

2.4.7 Module

```
Module driftdiffusion
{

    name = dd

    quadrature_rule = trapez

    plot = (Ec, Ev, ElPotential, ElField, eQFermi, hQFermi, eDensity, hDensity,
            eCurrentDensity, hCurrentDensity, CurrentDensity, ContactCurrents,
            NetRecombination, eMobility, hMobility,
            IonizedDonors, IonizedAcceptors,
            IonizedElectronTraps)
```

```
Solver linesearch
{
}
```

2.4.8 Physic

We'll use the Fermi-Dirac statistic:

```
Physics
{

    statistics = FD

    mobility doping_dependent {}

    recombination (srh, auger) {}

    trap eNeutral
    {
        regions = Si/SiO2
        Nt = 2e12
        Et = 0.5
        reference = cb
    }
}
```

Contacts

```
Contact base
{
    type = ohmic
    voltage = $Vb
}
```

```
Contact collector
{
    type = ohmic
    voltage = $Vc
}
```

```
Contact emitter
{
    type = ohmic
    voltage = 0.0
}
```


2.4.9 Sweep

We can calculate the output characteristics and plot only for the last collector bias:

Module sweep

```
{
  name = sweep_base
  variable = $Vb
  start = 0.5
  stop = 0.7
  steps = 10

  solve = sweep_coll

  plot_data = true
}
```

Module sweep

```
{
  name = sweep_coll
  variable = $Vc
  start = 0.0
  stop = 3
  steps = 60

  solve = dd
  plot_data = true
}
```

2.4.10 Simulation

The temperature = 300 is the default

Simulation

```
{
  verbose = 3

  solve = sweep_base

  resultpath = output2
  output_format = vtk
}
```


THERMAL

3.1 Boundary conditions

The Dirichlet boundary condition is set with

```
Contact reservoir
{
    type = heat_reservoir
    temperature = 300
}
```

whereas the surface thermal resistance is applied with

```
Contact dissipator
{
    r_surf = 0.5
    type = surface_resistance
    temperature = 300
}
```

The selfconsistent simulation solves the thermal and the drift diffusion simulations (named **tt** and **dd** respectively) in a self consistent way.

```
Module Selfconsistent
{
    flavour = relaxation
    simulations = (dd,tt)
    abs_tolerance = 1e-5
    monitor = true
}
```

The sweep section runs the selfconsistent simulation for each anode bias step

```
Module sweep
{
```

```
simulation = selfconsistent
variable = Vb
start = 0.0
stop = 1.0
steps = 10
}
Simulation
{
  temperature = 300
  solve = sweep
  symmetry = cylindrical
}
```

The temperature map, the heat source and the thermoelectri power are shown below.

```
Contact heat_reservoir
{
  region = substrate
  temperature = 300
}

Contact substrate
{
  type = surface_resistance
  r_surf = 0. 05
  temperature = 300
}
```

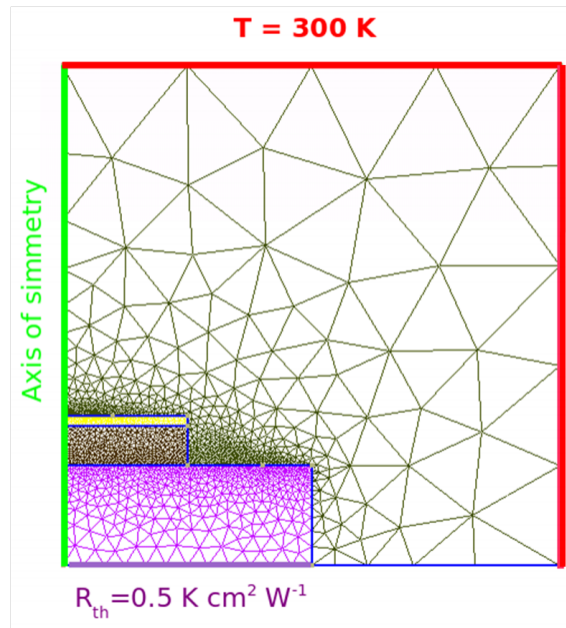
3.2 Heat sources

In this example we will see how to perform a thermal-driftdiffusion self-consistent simulation. We include the Seebeck and Peltier effects. The heat conduction through the environment is modelled by adding an air region all around the diode. We assume that far from the junction the system is in thermal equilibrium with a thermal bath at 300 K.

The heat dissipation through the substrate is taken into account by introducing a thermal surface resistance. Furthermore, we rely on the cylindrical symmetry with respect to the axis growth and consider only a 2D slice of the system. In this way we simulate a 3D device by performing a 2D simulation (with much less computational time). The mesh corresponding to the whole system (diode plus air region) is shown below.

Region n-side corresponds to the n-doped region situated at the bottom of the device, while the region p-side refers to the top p-doped region. Air is modelled with Region Air.

```
Device
{
```



```

meshfile = tut_09.msh
dimension = 2
material = Si
Region n_side
{
    doping = 1e18
    doping_type = donor
}
Region p_side
{
    doping = 1e19
    doping_type = acceptor
}
Region air
{
}
}

```

The drift diffusion simulation is performed only over the device domain (excluding the Air region). Therefore, in option subsection we have to indicate *physical_regions = (n side, p side)* . The mobility models used for both electrons and holes are doping dependent.

```

electron_mobility doping_dependent{}

hole_mobility doping_dependent{}

```

The connection with the thermal simulation is given by the keyword *thermal_simulation = tt* . The thermoelectric power model used can be included with *thermoelectric diffusivity model* (see

the **Theory** section). Direct bias applied to the diode is obtained by using the following boundary conditions

```
Contact anode{@Vb[0.0]}
```

```
Contact cathode{}
```

Unlike the drift diffusion case, the thermal simulation has to be performed over the whole simulation domain (including the Air region). We can refer to all regions simply by writing *physical_regions = all* . The connection with the drift diffusion simulation is specified with

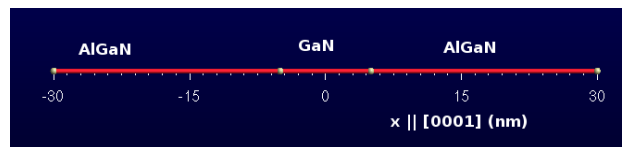
```
HeatSource joule
{
    DD_simulation_name = dd
}
```

```
HeatSource constant
{
    region = qdot
    H = 1e10
}
```

ENVELOPE FUNCTION APPROXIMATION

4.1 Quantized states in a GaN/AlGaN quantum well

We consider a GaN quantum well embedded into a $Al_{0.3}Ga_{0.7}N$ barrier. The structure is grown on a $Al_{0.3}Ga_{0.7}N$ substrate.



The structure geometry file is `quantum_well.geo` .

```
Point(1) = {-30, 0, 0, 1};
Point(2) = {-5, 0, 0, 0.1};
Point(3) = {5, 0, 0, 0.1};
Point(4) = {30, 0, 0, 1};
Line (1) = {1, 2};
Line (2) = {2, 3};
Line (3) = {3, 4};
Physical Line (&quot;well&quot;) = {2};
Physical Line (&quot;buffer1&quot;) = {1};
Physical Line (&quot;buffer2&quot;) = {3};
```

There are three physical regions (the well and two barriers, defined by means of Physical Lines 1, 2 and 3) and two physical points, at the simulation domain boundary, which will be associated to the boundary conditions (see *Tutorial Bulk Si* and *Tutorial Si-Pn Diode*).

We'll study the following properties:

- Strain
- Band profile

- Quantized states of electrons and holes

The Device section of the input file reads:

Device

```
{
  Region well
  {
    material = GaN
    structure = wz
    y-growth-direction = (1,0,-1,0)
    z-growth-direction = (-1,2,-1,0)
    x-growth-direction = (0,0,0,1)
    doping = 1e18
    doping_type = acceptor
    doping_level = 0.025
  }
  Region buffer1
  {
    material = AlGaIn
    x = 0.3
    structure = wz
    y-growth-direction = (1,0,-1,0)
    z-growth-direction = (-1,2,-1,0)
    x-growth-direction = (0,0,0,1)
    doping = 5e16
    doping_type = acceptor
    doping_level = 0.025
  }
  Region buffer2
  {
    material = AlGaIn
    x = 0.3
    structure = wz
    y-growth-direction = (1,0,-1,0)
    z-growth-direction = (-1,2,-1,0)
    x-growth-direction = (0,0,0,1)
    doping = 5e16
    doping_type = acceptor
    doping_level = 0.025
  }
}
```

4.1.1 Calculation of strain

The strain is calculated over all the structure. Since the structure is grown over a substrate, we define an appropriate boundary condition. The substrate boundary condition is associated with

Physical Point 1 . The Models section reads:

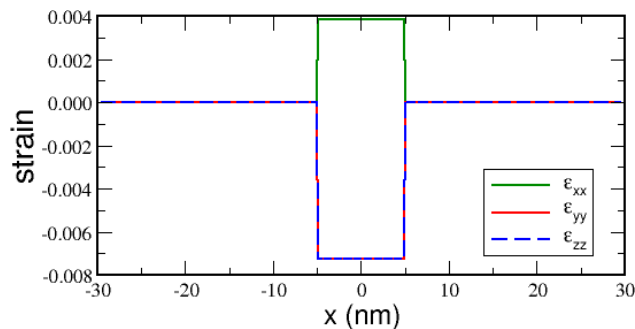
```
model macrostrain
{
  options
  {
    simulation_name = str
    physical_regions = all
  }
  BC_Regions
  {
    BC_Region substrate
    {
      type = substrate
      material = AlGaIn
      x = 0.3
      structure = wz
      y-growth-direction = (1,0,-1,0)
      z-growth-direction = (-1,2,-1,0)
      x-growth-direction = (0,0,0,1)
    }
  }
}
```

The Solver section defines that the structure has a substrate:

```
macrostrain
{
  substrate = substrate
}
```

In the output section we specify the variable name **strain** .

The results look as follows:



The well is strained, but the barriers are not, because the substrate material and the barrier material coincide.

4.1.2 Calculation of the band profile

The band profile calculation needs the Poisson equation to be solved. Therefore, we have to define a drift-diffusion model and solve it without any voltage applied. The Models section reads:

```
model driftdiffusion
{
  options
  {
    simulation_name = dd
    physical_regions = all
  }
}
```

This section does not contain any boundary conditions. It means that we rely on the *default* boundary condition, that require zero gradient of electric potential near the boundary. This boundary condition, known as the *flat band boundary condition* , is usually applied if the simulation domain represents a small part of a real heterostructure.

It assumes that the free charge accumulated somewhere outside of the simulation domain screens the electric field.

The Solver section reads:

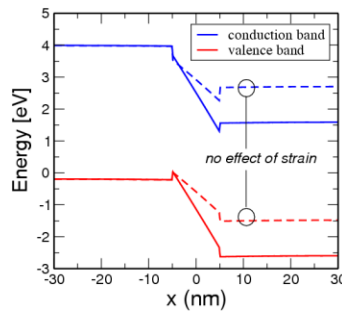
```
driftdiffusion
{
  coupling = poisson
  nonlin_max_it = 70
  nonlin_rel_tol = 1e-10
  nonlin_abs_tol = 1e-10
  nonlin_step_tol = 1e-9
  ls_max_step = 4
  discretization = fem
  integration_order = 2
  ksp_type = bcgs
}
```

The most important parameter is `coupling = poisson` that defines that no transport equation is solved. The rest of the parameters define properties of the numerical solver. The important thing is to define a physical model in the model Physics, that reads as follows:

```
driftdiffusion
{
  strain_simulation = str
}
```

Here we impose that the drift diffusion model takes into account both the deformation potential effect and the piezoelectric effect. The latter is very important for wurtzite materials. In order to plot the band diagram we ask for two variables to be written out, namely E_c , E_v . The band diagram

calculated with and without taking into account the effect of strain is shown:



4.1.3 Quantized states of electrons and holes

We are going to study quantized states of electrons and holes in the quantum well. Since the structure is 1D, the eigenstate is characterized by the energy level number n and the $k_{||}$ vector that is perpendicular to the growth direction. First, we define two simulations that solve Schrödinger for a single $\mathbf{k}$ -vector, for electrons and holes. In order to simplify the tutorial we solve the Schrödinger equation over all the structure.

The Models section reads:

```
model efashroedinger
{
  options
  {
    simulation_name = quantum_el
    physical_regions = all
  }
}
model efashroedinger
{
  options
  {
    simulation_name = quantum_hl
    physical_regions = all
  }
}
```

The Solver section reads as follows:

```
quantum_el
{
  Dirichlet_bc_everywhere = true
  particle = el
  number_of_eigenstates = 10
  poisson_model_name = dd
```

```
    strain_model_name = macrostrain
    solution_method = general
    solver = krylovshur
}
quantum_hl
{
    Dirichlet_bc_everywhere = true
    particle = hl
    number_of_eigenstates = 15
    poisson_model_name = dd
    strain_model_name = macrostrain
    solver = krylovshur
    solution_method = general
}
```

Here we defined two different option sets for electrons and holes. We specified that

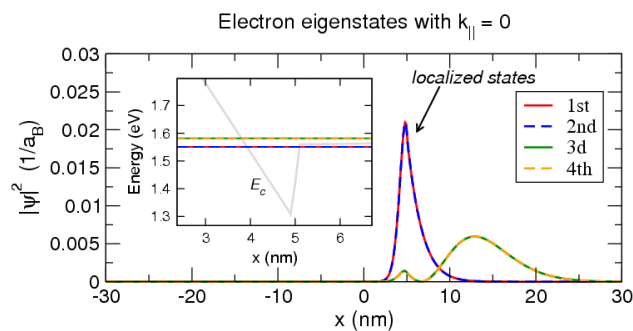
- Dirichlet boundary condition is imposed at the simulation domain boundary
- Electric potential and strain is applied

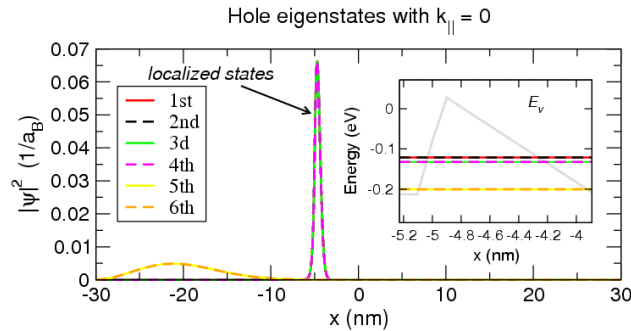
The Physics section defines the 8x8 $k \cdot p$ model for electrons and holes:

```
quantum_el
{
    particle = el
    model = kp
    kp_model = 8x8
}
quantum_hl
{
    particle = hl
    model = kp
    kp_model = 8x8
}
```

We choose to plot the following variables: EigenEnergy, EnergyLevels, EigenFunctions.

The results (levels and $|\Psi|^2$) for electrons and holes are shown:





Now we would like to study the dispersion of the two first electron and hole states. In this tutorial we show how to define different k-spaces and calculate the particle dispersion over them.

First, we would like to calculate the dispersion along a line in k-space. We define two simulations belonging to the model **quantumdispersion** :

```
dispersion1D_el and dispersion1D_hl .

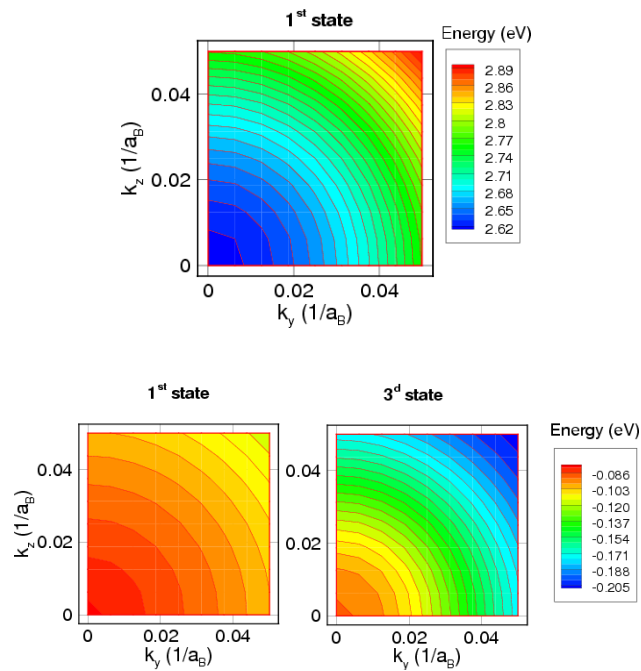
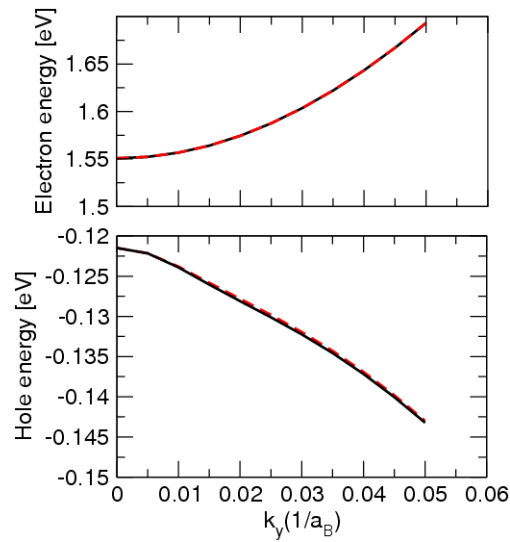
model quantumdispersion
{
  options
  {
    simulation_name = dispersion1D_el
    physical_regions = all
  }
}

model quantumdispersion
{
  options
  {
    simulation_name = dispersion1D_hl
    physical_regions = all
  }
}
```

We have to connect the **quantumdispersion** simulations to the related quantum (**efaschroedinger**) ones. Thus, in Solver section:

```
dispersion1D_el
{
  quantum_simulation = quantum_el
  wedge = half
  k_space_dimension = 1
  k1 = (0, 0.1, 0)
  number_of_nodes = (10)
  min_eigenvalue_number = 0
  max_eigenvalue_number = 1
}
```

```
dispersion1D_hl
{
  quantum_simulation = quantum_hl
  wedge = half
  k_space_dimension = 1
  k1 = (0, 0.1, 0)
  number_of_nodes = (10)
  min_eigenvalue_number = 0
  max_eigenvalue_number = 1
}
```



Attachment Size

- [quantum_well.geo](#) 352 bytes
- [tutorial6.tib](#) 4.53 KB

SIMULATION DYE SOLAR CELLS

In this Tutorial we will show how to model and simulate a Dye Sensitized Solar Cell (DSC).

The basic structure of a DSC is the following:

1. A photoanode made of a semiconductor mesoporous medium (TiO_2) plunged into a liquid electrolyte.
2. A region of liquid electrolyte only in contact with a counter-electrode covered by a thin platinum layer.

The surface of the semiconductor is covered by a mono-layer of dye molecules which make the oxide photoactive. The mesoporous material is obtained by sintering together nanoparticles of TiO_2 of 15-20 nm in diameter.

The diameter of the nanoparticles is fundamental to obtain the right porosity of the material, in fact in order to have a high light harvesting from the dye molecules is necessary to have a large effective area that can be obtained only with a porous material, moreover the porosity of the semiconductor allows the electrolyte to be in contact with the dye molecules.

The functioning of the device is the following: when a photon is absorbed by a dye, it excites an electron which is transferred into the conduction band of the porous material. The charge transfer is extremely fast, in the order of tens of fs.

Then, the electron percolates inside the porous material until it reaches the anode contact: a transparent conductive oxide (TCO). The ionized dye is regenerated by the electrolyte via a redox process where iodide is oxidized and triiodide is formed. This regeneration has two important consequences: on the one hand the dye is now ready to absorb another photon and on the other hand the equilibrium of the redox pair concentration in the electrolyte is broken. In particular, the concentration of triiodide is increased at the expense of iodide concentration.

A concentration gradient is formed that moves triiodide ions towards the counter-electrode and iodide ions towards the mesoporous material. Finally, the triiodide ions reach the platinum where they are reduced and transformed back in iodide ions. The cycle of charge carriers is completed and a current has been established inside the cell exploiting the energy of the photon absorbed.

Two important facts must be noted:

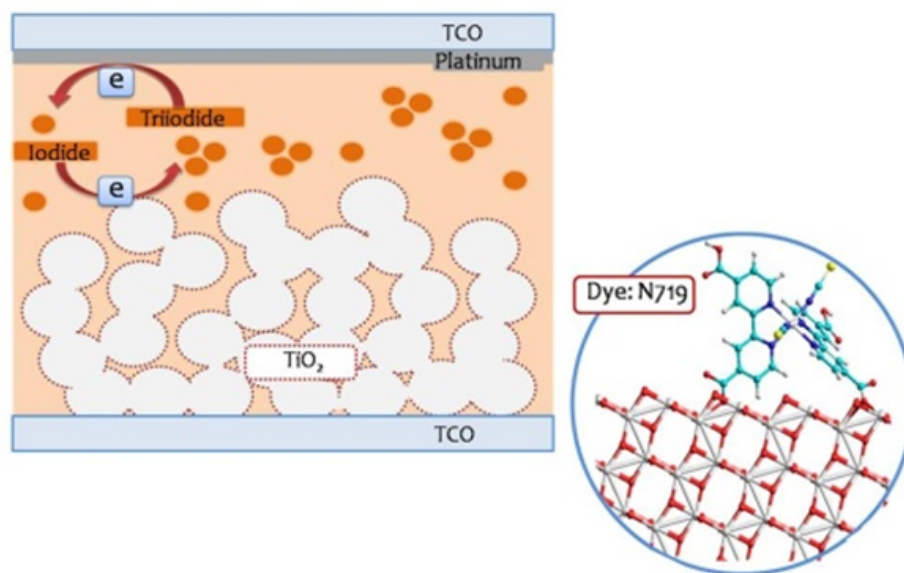


Figure 5.1: Fig. 1: Scheme of a DSC (in the inset: a typical Dye molecule).

1. In the cell there is no permanent chemical transformation;
2. The split of the electron-hole pair created by the photon occurs directly at the dye level, the electron percolates inside the TiO_2 while the hole is handled by the electrolyte. This means that charge carriers in DSCs are majority carriers and the recombination is suppressed. That is the reason why DSCs allow reasonable efficiencies despite they are made of very poor quality materials.

To run this tutorial the following files should be present in the working directory:

`dsc.tib` : input file (physical parameters of the device)

`dsc.msh` : mesh generated by GMSH from the script (geometry of the device)

`dsc.geo` : script for **GMSH**

5.1 Device structure

In this tutorial we will see a simple 1D model, made of two physical regions:

1. TiO_2 region, which is the mesoporous medium where the light absorption and recombination takes place
2. Electrolyte region

Two boundary condition points represent the photoanode (left) and the counter-electrode (right).

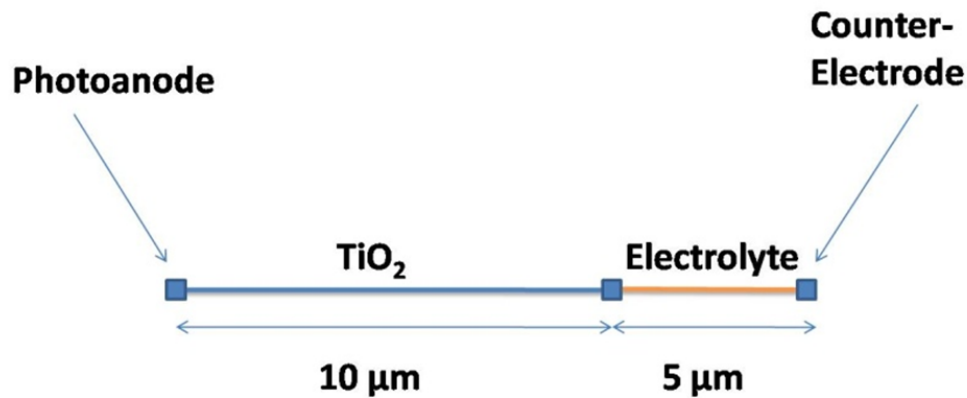


Fig. 2: The scheme of the 1D device.

Definition of dsc.tib

This is the description of the device physical regions:

```
Device
{
  meshfile = bulk.msh

  Region TiO2
  {
    material = TiO2mes
    porosity = 0.5
  }

  Region electrolyte
  {
    material = TiO2mes
    TiO2 = false
  }
}
```

[warn] for both region (:math:`\text{TiO}\_2` and electrolyte) the same material for the TiO2mes), the parameter porosity defines which porosity has the material. The flag TiO2 = false defines instead that in the electrolyte region only the elect is present and there is no semiconductor. By default the dssc module assumes that a contains both the semiconductor and the electrolyte.

This is the definition of Simulation Models and associated Boundary Conditions:

```
Module dssc
{
  name = dssc
```

```
plot = (Potential, Density, TotalChargeDensity, NetRecombination, Current, EField)

Solver linesearch
{
  max_iterations = 300
  step_tolerance = 1e-4
  linear_solver
  {
    preconditioner = lu
  }
}

Physics
{
  generation = dssc_generation
  beta = 1.0
}

Contact anode
{
  type = ohmic
  bias = $V[0.0]
}

Contact cathode
{
  type = Pt
  load = $R[1e8]
}
}
```

The module `dssc` contains the name of the simulation (and the list of output plotted), the subsections for the solver, the physics and the contacts. The physics subsection must contain at least one flag for the generation module. The non linear exponent in the electron density recombination is assumed to 1 (linear case), but can be a changed to a different value for non-linear recombination. The contact modules must be two: photoanode and counter-electrode.

The two contacts are `type = ohmic` (photoanode) and `Pt` (counter-electrode). The external bias is applied at the photoanode (bias), while the counter-electrode is closed over an external resistance (load). In case it is wanted to simulate a cell under illumination the load must be set to a high value (open-circuit condition) and then switched to a `load = 1` value (short-circuit condition).

For a simulation in dark condition, load can be set to `load = 1` immediately.

The contact modules can include the flag `region = (region_1, region_#)` in case that contact (cathode or anode) is the composition of different mesh regions.

```

Module dssc_generation
{
    regions = TiO2
    plot = (Distance, Generation)
    light_direction = (1, 0, 0)
    light_intensity = $x
    dye = N719

    Contact anode
    {
    }
}

```

The Module **dssc_generation** defines the photogeneration within the device. The generation model must define which is the region(s) where photogeneration occurs (regions), the intensity of light (light_intensity), the direction of the light rays (light_direction), the kind of Dye used in the cell (dye) and the boundary region where light has the maximum intensity (in our example “anode”).

Another flag illumination_spectrum = <material> can be set if we want to change the solar spectrum for generation (for example in case the light source we want to simulate is not the conventional sun spectrum).

By default the flag is set to illumination_spectrum = Sun1p5am conventional sun spectrum 1.5 AM.

```

Module sweep
{
    name = sweep_gen
    solve = (dssc_generation, dssc)
    variable = $x
    values = (0, 1e-12, 1e-11, 1e-10, 1e-9, 1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2)
    plot_data = true
}

```

```

Module sweep
{
    name = sweep_R
    solve = dssc
    variable = $R
    values = ( 1000, 100, 10, 1 )
    plot_data = true
}

```

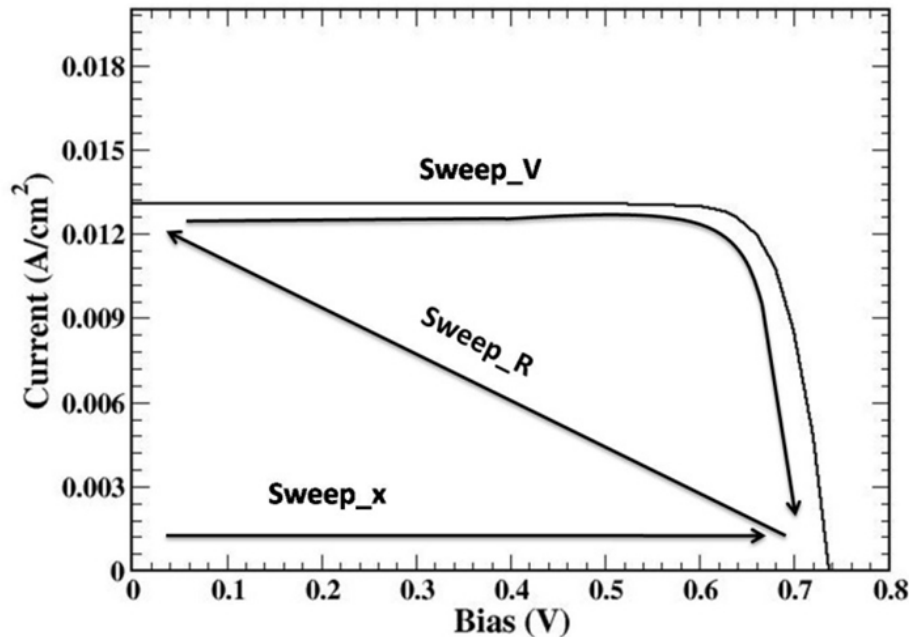
```

Module sweep
{
    name = sweep_V
    solve = dssc
    variable = $V
}

```

```
values = (0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.62, 0.64, 0.66, 0.68, 0.7, 0.72, 0.74, 0.76, 0.78, 0.8)
plot_data = true
}
```

The three sweeps needed to solve the cell: the first sweep is needed to push the device to open circuit condition under illumination, the second sweep switches the state of the cell from open circuit condition (high external load) to short circuit condition (load = #), then the final sweep for the bias. In figure the three sweeps of the code are shown:



The definition of model-independent parameters of the Simulation:

```
Simulation
{
  searchpath = ../../materials
  solve = (sweep_gen, sweep_R, sweep_V)
  resultpath = output
  output_format = grace
}
```

The **output** of the calculation is set in the module dssc section.

We can chose to plot the internal potential profiles (flag Potential) which includes electrostatic potential, all the densities (flag Density) which includes electron, iodide, triiodide, cation densities and recombination and generation profiles, all the currents (flag Current) which include the x, y and z components of the currents (electronic, iodide, triiodide and cation, total current) and the electric field.

Finally the flag (ContactCurrents) allows the plot of the current flowing through the contacts.



In a DSC the current I_s is fundamentally diffusion driven, the electric field and the consequent drift current is rather small.

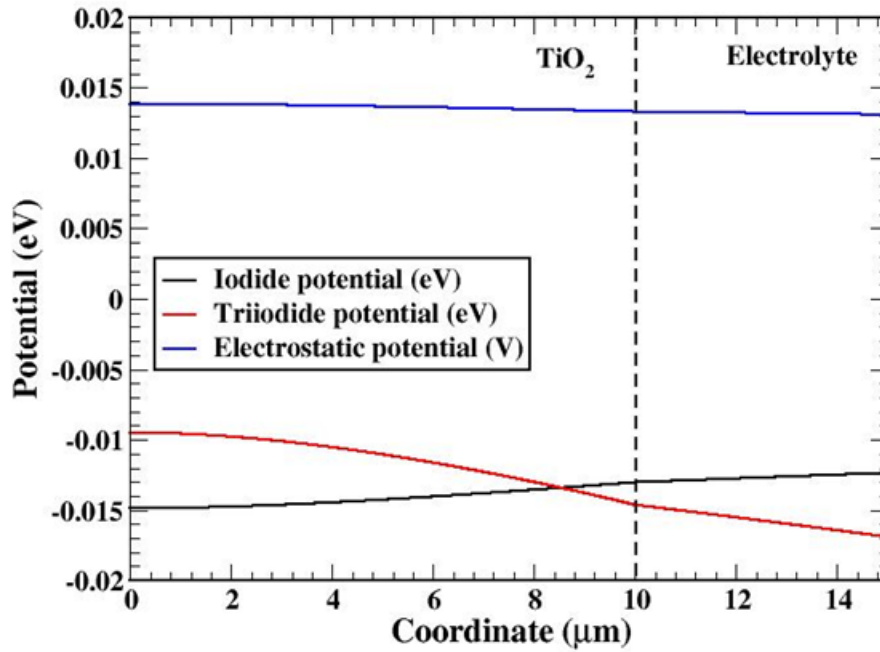


Figure 5.2: Fig. 4: Potential profiles for the ionic species and the electrostatic potential (short-circuit condition).

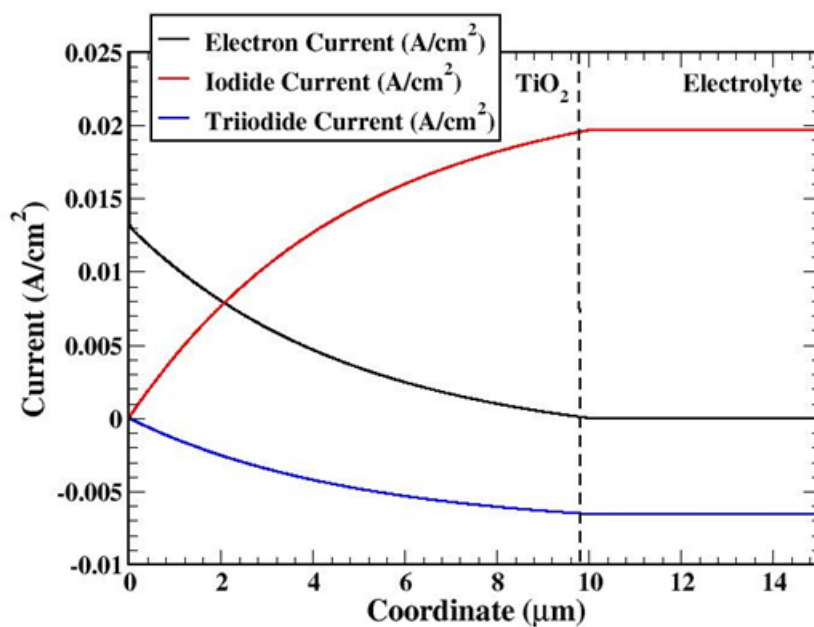


Figure 5.3: Fig. 5: Current contributions within the cell (short-circuit condition).

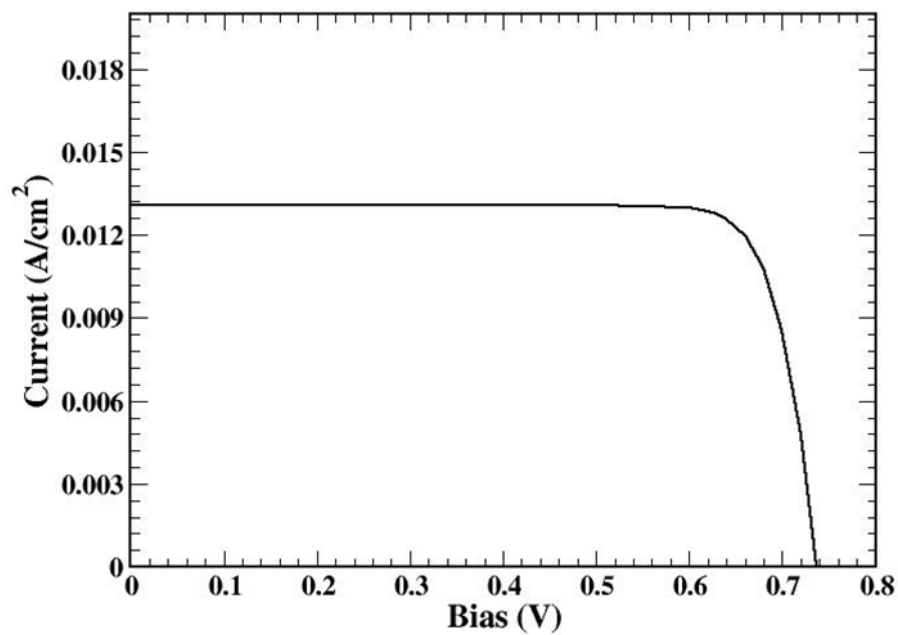


Figure 5.4: Fig. 6: I-V characteristic of the device.

Part II

Glossary

abs_tolerance

Level: paragraph

Description:

Example:

`abs_tolerance = 1e-3`

analytic

Level: paragraph

Description: { true or false } If true then the density is calculated analytically, otherwise numerically.

Example:

`analytic = true`

area_factor

Level: paragraph

Description:

Example:

`area_factor = 0.1`

barrier_height

Level: paragraph

Description:

Example:

`barrier_height = 3.0`

beta

Level: paragraph

Description:

Example:

beta = 1.0

bias

Level: paragraph

Description:

Example:

bias = @V[0.0]

Contact/anode

Level: Block

Description:

Example:

```
Contact anode
{
    type = ohmic
    voltage = $Vb[0.0]
}
```

Contact/backcontact

Level: Block

Description:

Example:

```
Contact backcontact
{
    voltage = 0.0
    area_factor = 0.1
}
```

Contact/Base

Level: Block

Description:

Example:

```
Contact Base
{
    type = clamp
}
```

Contact/cathode

Level: Block

Description:

Example:

```
Contact cathode
{
    type = ohmic
    voltage = 0.0
}
```

Contact/dissipator

Level: Block

Description:

Example:

```
Contact dissipator
{
    r_surf = 0.5
    type = surface_resistance
    temperature = 300
}
```

Contact/gate

Level: Block

Description:

Example:

```
Contact gate
{
    regions = (gate1, gate2)
    type = schottky
    work_function = 1.5272
    voltage = @Vg
}
```

Contact/heat_reservoir

Level: Block

Description:

Example:

```
Contact heat_reservoir
{
    region = substrate
    temperature = 300
}
```

Contact/left

Level: Block

Description:

Example:

```
Contact left
{
    type = heat_reservoir
    temperature = 300
}
```

Contact/reservoir

Level: Block

Description:

Example:

```
Contact reservoir
{
    temperature = 300
}
```

Contact/right

Level: Block

Description:

Example:

```
Contact right
{
    type = heat_reservoir
    temperature = 300
}
```

Contact/substrate

Level: Block

Description:

Example:

```
Contact substrate
{
    type = surface_resistance
    r_surf = 0. 05
    temperature = 300
}
```

Contact/surface_resistance

Level: Block

Description:

Example:

```
Contact surface_resistance
{
    r_surf = 0.01
    temperature = 300
}
```

Contact/Top

Level: Block

Description:

Example:

```
Contact Top
{
```



```
        type = surface_force
        force = (0.0, 0.0625,0.0)
    }
```

coupling

Level: paragraph

Description:

Example:

```
coupling = electrons
```

dd_simulation

Level: paragraph

Description: The name of the drift-diffusion simulation is given by dd_simulation.

Example:

```
dd_simulation = dd
```

dE

Level: paragraph

Description:

Example:

```
dE = 0.001
```

degeneracy

Level: paragraph

Description: degeneracy of the quantum state

Example:

degeneracy = 2

density

Level: paragraph

Description: doping concentration [cm-3]

Example:

density = 1e17

dimension

Level: paragraph

Description:

Example:

dimension = 3

Dirichlet_bc_everywhere

Level: paragraph

Description:

Example:

Dirichlet_bc_everywhere = true

discretization

Level: paragraph

Description:

Example:

discretization = fem

Doping

Level: sub-Block

Description:

Example:

```
Doping
{
    Nd = 1e16
    type = donor
}
```

doping_level

Level: paragraph

Description:

Example:

doping_level = 0.035

doping_type

Level: paragraph

Description:

Example:

doping_type = donors

dye

Level: paragraph

Description: the kind of dye used in the cell

Example:

dye = N719

E_{max}

Level: paragraph

Description:

Example:

E_{max} = 3.6

E_{min}

Level: paragraph

Description:

Example:

E_{min} = 2.8

final_eigenstates

Level: paragraph

Description: range of eigenstates to be taken in account for optical calculations.

Example:

final_eigenstates = (0, 15)

final_state_model

Level: paragraph

Description:

Example:

```
final_state_model = quantum_hl
```

first_state

Level: paragraph

Description:

Example:

```
first_state = 3
```

flavour

Level: paragraph

Description:

Example:

```
flavour = relaxation
```

force

Level: paragraph

Description:

Example:

```
force = (0.0, 0.0625,0.0)
```

generation

Level: paragraph

Description:

Example:

generation = dssc_generation

H

Level: paragraph

Description:

Example:

H = 1e10

initial_eigenstates

Level: paragraph

Description: range of eigenstates to be taken in account for optical calculations.

Example:

initial_eigenstates = (0, 9)

initial_eigenstates_number

Level: paragraph

Description: initial number of eigenstates for the Schroedinger equation

Example:

initial_eigenstates_number = 10

initial_state_model

Level: paragraph

Description:

Example:

```
initial_state_model = quantum_el
```

integration_order

Level: paragraph

Description:

Example:

```
integration_order = 2
```

k-space_basis

Level: paragraph

Description:

Example:

```
k-space_basis = true
```

k-space_dimension

Level: paragraph

Description:

Example:

```
k-space_dimension = 0
```

k1

Level: paragraph

Description:

Example:

$k1 = (0, 0, 0.1)$

k2

Level: paragraph

Description:

Example:

$k2 = (0, 0.1, 0)$

kp_model

Level: paragraph

Description:

Example:

$kp_model = 8 \times 8$

ksp_type

Level: paragraph

Description:

Example:

$ksp_type = ponly$ #preconditioner only

level

Level: paragraph

Description: energy level of the dopant [eV]

Example:

level = 0.025

light_direction

Level: paragraph

Description:

Example:

light_direction = <vector Indicating the direction of light>

light_intensity

Level: paragraph

Description: the vector which fixes the direction from where the light comes.

Example:

light_intensity = @x

load

Level: paragraph

Description:

Example:

load = @R[1e8]

low_field_model

Level: paragraph

Description:

Example:

low_field_model = doping_dependent

ls_max_step

Level: paragraph

Description:

Example:

ls_max_step = 4

mat

Level: paragraph

Description:

Example:

mat = TiO2mes

material

Level: paragraph

Description:

Example:

material = Si

max_eigenvalue_number

Level: paragraph

Description: the dispersion calculated for the states number i

Example:

`max_eigenvalue_number = 5`

max_iterations

Level: paragraph

Description:

Example:

`max_iterations = 5`

mesh_order

Level: paragraph

Description:

Example:

`mesh_order = FIRST`

mesh_regions

Level: paragraph

Description:

Example:

`mesh_regions = (barrier_dop_s, barrier_dop_d)`

mesh_units

Level: paragraph

Description:

Example:

`mesh_units = 1e-6`

meshfile

Level: Block

Description:

Example:

`meshfile = bulk.msh`

min_eigenvalue_number

Level: paragraph

Description: the dispersion calculated for the states number i

Example:

`min_eigenvalue_number = 0`

model

Level: paragraph

Description:

Example:

`model = conduction_band`

Modules/BodyForceconverse_piezo

Level: Block

Description:

Example:

```
BodyForceconverse_piezo
{
    poisson_simulation = dd
}
```

Modules/BodyForcelattice_mismatch

Level: Sub-Block

Description:

Example:

```
BodyForcelattice_mismatch
{
    x-growth-direction = (-1,0,1,0)
    y-growth-direction = (-1,2,-1,0)
    z-growth-direction = (0,0,0,1)
    reference_material = AlGaN
    structure = wz
    x = 0.2
}
```

Modules/Boundary

Level: Sub-Block

Description:

Example:

```
Boundary substrate
{
    regions = bottom
}
```

```
        material = GaN
        y-growth-direction = (0,0,0,1)
        z-growth-direction = (1,0,-1,0)
        x-growth-direction = (-1,2,-1,0)
    }
```

Modules/Cluster

Level: Block

Description:

Example:

```
Cluster semiconductor
{
    regions = -passivation
}
```

Modules/Device

Level: Super-Block

Description:

Example:

```
Device
{
    meshfile = bulk.msh
    Region bulk
    {
        material = Si
        Doping
        {
            Nd = 1e16
            type = donor
        }
    }
}
```

Modules/driftdiffusion

Level: Super-Block

Description:

Example:

```
Module driftdiffusion
{
    plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)

    Contact anode { voltage = $Vb }
    Contact cathode { }
}
```

Modules/dssc

Level: Block

Description:

Example:

```
model dssc
{
    options
    {
        simulation_name = <name_of_the_model>
        plot(Potential, Density, Current, ContactCurrents)
    }
    ...
}
```

Modules/dssc_generation

Level: Block

Description:

Example:

```
model dssc_generation
{
  options
  {
    regions = (<TiO2_region_name_1>, <TiO2_region_name_2>, ...)
    plot = (Distance, Generation)
    light_direction = <vector Indicating the direction of light>
    (example, illumination from x direction: light_direction = (1, 0, 0))
    light_intensity = @x
    dye = N719
  }
  ...
}
```

Modules/efaschroedinger

Level: Block

Description:

Example:

```
Module efaschroedinger
{
  particle = el
  poisson_model_name = dd
  strain_model_name = strain # macrostrain
  name = quantum_el
  regions = quantum
  plot = (EigenFunctions, EigenEnergy, EnergyLevels)
  Solver
  { number_of_eigenstates = 10 # 30 }
  Physics
  { model = conduction_band }
```

Modules/elasticity

Level: Super-Block

Description:

Example:

```
Module elasticity
{
  Physics
  {
    Stiffness isotropic
    {
      Young = 129 Poisson = 0.349
    }
    Contact Base
    {
      type = clamp
    }
    Contact Top
    {
      type = surface_force force = (0.0, 0.0625,0.0)
    }
  }
}
```

Modules/HeatSource

Level: Sub-Block

Description:

Example:

```
HeatSource constant
{
  regions = HotSpot
  H = 1e10
}
```

Modules/LinearSolver

Level: Sub-Block

Description:

Example:

```
LinearSolver petsc
{
    tolerance = 1e-7
    max_iterations = 10000
    #xmonitor = true
    pc = composite
}
```

Modules/mobility

Level: Sub-Block

Description:

Example:

```
mobility field_dependent
{
    low_field_model = doping_dependent
}
```

Modules/NonlinearSolver

Level: Block

Description:

Example:

```
NonlinearSolver linesearch
{
    relative_tolerance = 1e-15
    step_tolerance = 5e-3
    max_iterations = 15
}
```

Modules/opticalspectrum

Level: Block

Description:

Example:

```
Module opticalspectrum
{
    k_space_dimension = 2
    k-space_basis = true
    k1 = (0, 0, 0.1)
    k2 = (0, 0.1, 0)
    refine_fraction = 0.30
    relative_accuracy = 0.01
    refine_k_space = true
    number_of_nodes = (2, 2)
    wedge = quarter
    plot = (optical_spectrum)
    optical_matr_elem_model = opticskp
    polarization = (0, 0, 1)
    Emin = 3.0
    Emax = 5.0
    dE = 0.001
}
```

Modules/opticskp

Level: Block

Description:

Example:

```
Module opticskp
{
    name = optics
    regions = quantum
}
```

```
    plot = (optical_spectrum_k_0 )
    initial_state_model = quantum_el
    final_state_model = quantum_hl
    initial_eigenstates = (0, 9)
    final_eigenstates = (0, 15)
    polarization = (0, 0, 1)
    Emin = 2.8
    Emax = 3.6
    dE = 0.001
}
```

Modules/particle_density

Level: Sub-Block

Description:

Example:

```
particle_density
{
    statistics = boltzmann
}
```

Modules/Physics

Level: Block

Description:

Example:

```
Physics
{
    particle_density
    {
        statistics = boltzmann
    }
    mobility field_dependent
    {
```

```
        low_field_model = doping_dependent
    }
    recombination (srh, auger) {}
}
```

Modules/quantumdensity

Level: Block

Description:

Example:

```
Module quantumdensity
{
    name = dens_el
    regions = quantum
    plot = quantum_density
    k-space_dimension = 0
    k-space_basis = true
    k1 = (0, 0, 0.1)
    quantum_simulation = quantum_el
    degeneracy = 2
    refine_fraction = 0.20
    relative_accuracy = 0.01
    refine_k_space = true
    output_density_in_k_space = true
    uniform_refinement = false
    mesh_order = FIRST
    first_state = 3
    initial_eigenstates_number = 10
    analytic = true
}
```

Modules/quantumdispersion

Level: Block

Description:

Example:

```
Module quantumdispersion
{
    simulation_name = dispersion1D_el
    regions = all
    quantum_simulation = quantum_el
    min_eigenvalue_number = 0
    max_eigenvalue_number = 5
    wedge = half
    k_space_dimension = 1
    k1 = (0, 0.1, 0)
    number_of_nodes = (10)
    output_format = grace
    plot = k-space_dispersion
}
```

Modules/Selfconsistent

Level: Block

Description:

Example:

```
Module Selfconsistent
{
    flavour = relaxation
    simulations = (dd,tt)
    abs_tolerance = 1e-5
    monitor = true
}
```

Modules/Solver

Level: Super-Block

Description:

Example:

```
Solver
{
    nonlinear_solver = tiber
    nonlin_max_it = 15
    nonlin_rel_tol = 1e-12
    nonlin_step_tol = 1e-5
}
```

Modules/Stiffness

Level: Block

Description:

Example:

```
Stiffness isotropic
{
    Young = 129
    Poisson = 0.349
}
```

Modules/sweep

Level: Super-Block

Description:

Example:

```
Module sweep
{
    solve = driftdiffusion
    variable = $Vb
    start = 0.0
    stop = 1
    steps = 10
    plot_data = true
}
```

Modules/sweep_gen

Level: Block

Description:

Example:

```
sweep_gen
{
    simulation = (dssc_generation, dssc)
    variable = x
    values = (0, 1e-9, 1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1)
    plot_data = true
}
```

Modules/trap

Level: Sub-Block

Description:

Example:

```
trap fixed_charge
{
    region = GaN/SiN
    Nt = 2.74e13
}
```

monitor

Level: paragraph

Description:

Example:

```
monitor = true
```


name

Level: paragraph

Description:

Example:

name = dd

Nd

Level: paragraph

Description:

Example:

Nd = 1e16

nonlin_abs_tol

Level: paragraph

Description:

Example:

nonlin_abs_tol = 1e-10

nonlin_max_it

Level: paragraph

Description:

Example:

nonlin_max_it = 15

nonlin_rel_tol

Level: paragraph

Description:

Example:

`nonlin_rel_tol = 1e-12`

nonlin_step_tol

Level: paragraph

Description:

Example:

`nonlin_step_tol = 1e-5`

nonlinear_solver

Level: paragraph

Description:

Example:

`nonlinear_solver = tiber`

number_of_eigenstates

Level: paragraph

Description: defines the number of eigenvalues and eigenfunctions to be found.

Example:

`number_of_eigenstates = 10 # 30`

number_of_nodes

Level: paragraph

Description: number of elements in k mesh, along each direction.

Example:

```
number_of_nodes = (10)
number_of_nodes = (2, 2)
```

optical_matr_elem_model

Level: paragraph

Description: name of the opticskp model associated

Example:

```
optical_matr_elem_model = opticskp
```

output_density_in_k_space

Level: paragraph

Description:

Example:

```
output_density_in_k_space = true
```

output_format

Level: paragraph

Description:

Example:

output_format = grace

particle

Level: paragraph

Description:

Example:

particle = el

plot

Level: paragraph

Description:

Example:

plot = (Ec, Ev, Eg, ElField, ElPotential, eQFermi, hQFermi, eDensity, hDensity,
NetRecombination, eCurrentDensity, hCurrentDensity, CurrentDensity,
ContactCurrent, eMobility, hMobility, IonizedDonors, IonizedAcceptors)
plot = all

plot_data

Level: paragraph

Description:

Example:

plot_data = true

Poisson

Level: paragraph

Description:

Example:

Poisson = 0.349

poisson_model_name

Level: paragraph

Description:

Example:

poisson_model_name = dd

poisson_simulation

Level: paragraph

Description:

Example:

poisson_simulation = dd

polarization

Level: paragraph

Description:

Example:

polarization (piezo, pyro) {}

polarization

Level: paragraph

Description:

Example:

polarization = (0, 0, 1)

porosity

Level: paragraph

Description:

Example:

porosity = 0.5

preconditioner

Level: paragraph

Description:

Example:

preconditioner = lu

quadrature_rule

Level: paragraph

Description:

Example:

quadrature_rule = trapez

quantum_simulation

Level: paragraph

Description: name of the efaschroedinger simulation.

Example:

```
quantum_simulation = quantum_el
```

r_surf

Level: paragraph

Description:

Example:

```
r_surf = 0. 05
```

recombination

Level: paragraph

Description:

Example:

```
recombination (srh, auger) { }
```

reference

Level: paragraph

Description:

Example:

```
reference = cb
```

reference_material

Level: paragraph

Description:

Example:

reference_material = AlGaN

refine_fraction

Level: paragraph

Description:

Example:

refine_fraction = 0.20

refine_k_space

Level: paragraph

Description:

Example:

refine_k_space = true

region

Level: paragraph

Description:

Example:

region = GaN/SiN

region = buffer

Region/Air

Level: Block

Description:

Example:

```
Region Air
{
    material = Air
}
```

Region/barrier

Level: Block

Description:

Example:

```
Region barrier
{
    material = AlGaIn
    x = 0.14
}
```

Region/barrier_doped

Level: Block

Description:

Example:

```
Region barrier_doped
{
    mesh_regions = (barrier_dop_s, barrier_dop_d)
    material = AlGaIn
    x = 0.14
    Doping
    {
        density = 1e21
        type = donor
    }
}
```

```
        level = 0.026
    }
}
```

Region/buffer_doped

Level: Block

Description:

Example:

```
Region buffer_doped
{
    mesh_regions = (buffer_dop_s, buffer_dop_d)
Doping
{
    density = 1e21
    type = donor
    level = 0.026
}
}
```

Region/bulk

Level: Block

Description:

Example:

```
Region bulk
{
    material = Si
Doping
{
    Nd = 1e16
    type = donor
}
}
```

Region/cap

Level: Block

Description:

Example:

```
Region cap
{
  Doping
  {
    density = 5e18
    type = donor
    level = 0.026
  }
}
```

Region/cap_doped

Level: Block

Description:

Example:

```
Region cap_doped
{
  mesh_regions = (cap_dop_s, cap_dop_d)
  Doping
  {
    density = 1e21
    type = donor
    level = 0.026
  }
}
```

Region/collector

Level: Block

Description:

Example:

```
Region coll # the collector region
{
    type = donor
    Nd = 5e15
}
```

Region/electrolyte

Level: Block

Description:

Example:

```
Region <name of the electrolyte region>
{
    mat = TiO2mes
    TiO2 = false
}
```

Region/n_side

Level: Block

Description:

Example:

```
Region n_side
{
    Doping
    {
        Nd = 1e18
        type = donor
    }
}
```

```
}
```

Region/Nanowire

Level: Block

Description:

Example:

```
Region Nanowire
{
    doping_type = donors
    doping_level = 0.035
}
```

Region/oxide

Level: Block

Description:

Example:

```
Region oxide
{
    material = SiO2
}
```

Region/p_side

Level: Block

Description:

Example:

```
Region p_side
{
    Doping
```

```
{  
    Nd = 1e18  
    type = acceptor  
}  
}
```

Region/passivation

Level: Block

Description:

Example:

```
Region passivation  
{  
    mesh_regions = (passiv1, passiv2, passiv3, passiv4)  
    material = SiN  
}
```

Region/porous

Level: Block

Description:

Example:

```
Region <name of the porous region>  
{  
    mat = TiO2mes  
    porosity = 0.5  
}
```

Region/Well

Level: Block

Description:

Example:

```
Region Well
{
    material = GaN
}
```

regions

Level: paragraph

Description:

Example:

```
regions = all #default to not specify
regions = -passivation #apply to all regions except passivation
```

rel_tolerance

Level: paragraph

Description:

Example:

```
rel_tolerance = 1e-6
```

relative_accuracy

Level: paragraph

Description:

Example:

```
relative_accuracy = 0.01
```

relative_tolerance

Level: paragraph

Description:

Example:

relative_tolerance = 1e-15

resultpath

Level: paragraph

Description:

Example:

resultpath = output

save_state

Level: paragraph

Description:

Example:

save_state = true

searchpath

Level: paragraph

Description:

Example:

searchpath = ../../materials

simulation_name

Level: paragraph

Description:

Example:

```
simulation_name = dispersion1D_el
```

solution_method

Level: paragraph

Description:

Example:

```
solution_method = general
```

solve

Level: paragraph

Description:

Example:

```
solve = driftdiffusion  
solve = sweep
```

solver

Level: paragraph

Description:

Example:

```
solver = krylovshur
```

start

Level: paragraph

Description:

Example:

start = 0.0

statistics

Level: paragraph

Description:

Example:

statistics = boltzmann

statistics = FermiDirac

step_tolerance

Level: paragraph

Description:

Example:

step_tolerance = 5e-3

steps

Level: paragraph

Description:

Example:

steps = 10

stop

Level: paragraph

Description:

Example:

stop = 1

strain_model_name

Level: paragraph

Description:

Example:

strain_model_name = strain # macrostrain

strain_simulation

Level: paragraph

Description:

Example:

strain_simulation = macrostrain

strain_simulation = elasticity

structure

Level: paragraph

Description:

Example:

structure = wz

symmetry

Level: paragraph

Description:

Example:

symmetry = cylindrical

temperature

Level: paragraph

Description:

Example:

temperature = 300

TiO2

Level: paragraph

Description:

Example:

TiO2 = false

type

Level: paragraph

Description:

Example:

type = donor

type = acceptor

type = field_dependent

type = heat_reservoir

```
type = surface_resistance  
type = surface_force
```

uniform_refinement

Level: paragraph

Description:

Example:

```
uniform_refinement = false
```

values

Level: paragraph

Description:

Example:

```
values = ( 1000, 100, 1)
```

variable

Level: paragraph

Description:

Example:

```
variable = $Vb
```

verbose

Level: paragraph

Description:

Example:

verbose = 2

wedge

Level: paragraph

Description: to reduce calculation time, by exploiting symmetry.

Example:

wedge = half

wedge = quarter

x

Level: paragraph

Description: the elementum percentage in the alloy

Example:

material = AlGaIn

x = 0.14

x-growth-direction

Level: paragraph

Description:

Example:

x-growth-direction = (0,0,0,1)

y-growth-direction

Level: paragraph

Description:

Example:

y-growth-direction = (1,0,-1,0)

Young

Level: paragraph

Description:

Example:

Young = 129

z-growth-direction

Level: paragraph

Description:

Example:

z-growth-direction = (-1,2,-1,0)

BIBLIOGRAPHY

- [Kalyanasundaram] Kalyanasundaram Kuppaswamy, “Dye-Sensitized Solar Cells”, *Editor Crc Pr Inc.* , 2010.
- [Gagliardi] Alessio Gagliardi, Matthias Auf der Maur, Desiree Gentilini and Aldo Di Carlo, “Modeling of Dye sensitized solar cells using a finite element method”, *J. Computational Electronics* , vol. 8, pp. 12, 2009.
- [Wachutka] Wachutka, IEEE Trans CAD Cicui. Integr and Syst. (1990)].
- [Povolotskiy] Povolotskiy et al. JAP (2006).
- [Wang] Wang et al. Science (2006).
- [Gao] Gao et al NanoLetter (2009).
- [Selberherr] Siegfried Selberherr, Analysis and Simulation of Semiconductor Devices, SpringerVerlag Wien New York, 1st edition, 1984.
- [Wachutka] Gerhard K. Wachutka, Rigorous thermodynamic treatment of heat generation and conduction in semiconductor device modeling,” IEEE Transaction on Computer-aided Design., vol. 9, pp. 11, 1990.

INDEX

A

abs_tolerance, 63
analytic, 63
area_factor, 63

B

barrier_height, 63
beta, 63
bias, 64
Bulk Si
 input, 10
 meshing, 9
 modeling, 7
 output, 13
 run, 12

C

characteristic
 n-MosFET 2D, 25
Contact/anode, 64
Contact/backcontact, 64
Contact/Base, 65
Contact/cathode, 65
Contact/dissipator, 65
Contact/gate, 66
Contact/heat_reservoir, 66
Contact/left, 66
Contact/reservoir, 67
Contact/right, 67
Contact/substrate, 68
Contact/surface_resistance, 68
Contact/Top, 68
coupling, 69

D

dd_simulation, 69
dE, 69
degeneracy, 69
density, 70
device
 n-MosFET 2D, 23
 Si-Pn Diode, 15
dimension, 70
Dirichlet_bc_everywhere, 70
discretization, 70
Doping, 71
doping_level, 71
doping_type, 71
dye, 71

E

E_{max}, 72
E_{min}, 72

F

final_eigenstates, 72
final_state_model, 72
first_state, 73
flavour, 73
force, 73

G

generation, 73

H

H, 74

I

initial_eigenstates, 74

initial_eigenstates_number, 74

initial_state_model, 74

input

 Bulk Si, 10

integration_order, 75

K

k-space_basis, 75

k-space_dimension, 75

k1, 75

k2, 76

kp_model, 76

ksp_type, 76

L

level, 76

light_direction, 77

light_intensity, 77

load, 77

low_field_model, 77

ls_max_step, 78

M

mat, 78

material, 78

max_eigenvalue_number, 78

max_iterations, 79

mesh_order, 79

mesh_regions, 79

mesh_units, 79

meshfile, 80

meshing

 Bulk Si, 9

min_eigenvalue_number, 80

model, 80

 Si-Pn Diode, 16

modeling

 Bulk Si, 7

Modules/BodyForceconverse_piezo, 80

Modules/BodyForcelattice_mismatch, 81

Modules/Boundary, 81

Modules/Cluster, 82

Modules/Device, 82

Modules/driftdiffusion, 83

Modules/dssc, 83

Modules/dssc_generation, 83

Modules/efaschroedinger, 84

Modules/elasticity, 84

Modules/HeatSource, 85

Modules/LinearSolver, 85

Modules/mobility, 86

Modules/NonlinearSolver, 86

Modules/opticalspectrum, 87

Modules/opticskp, 87

Modules/particle_density, 88

Modules/Physics, 88

Modules/quantumdensity, 89

Modules/quantumdispersion, 89

Modules/Selfconsistent, 90

Modules/Solver, 90

Modules/Stiffness, 91

Modules/sweep, 91

Modules/sweep_gen, 92

Modules/trap, 92

monitor, 92

N

n-MosFET 2D

 characteristic, 25

 device, 23

 output, 28

 physic, 27

 simulation, 24

name, 93

Nd, 93

nonlin_abs_tol, 93

nonlin_max_it, 93

nonlin_rel_tol, 94

nonlin_step_tol, 94

nonlinear_solver, 94

number_of_eigenstates, 94

number_of_nodes, 95

O

optical_matr_elem_model, 95

output

 Bulk Si, 13

 n-MosFET 2D, 28

 Si-Pn Diode, 21

output_density_in_k_space, 95

output_format, 95

P

particle, 96

physic

 n-MosFET 2D, 27

 Si-Pn Diode, 19

plot, 96

plot_data, 96

Poisson, 96

poisson_model_name, 97

poisson_simulation, 97

polarization, 97

porosity, 98

preconditioner, 98

Q

quadrature_rule, 98

quantum_simulation, 98

R

r_surf, 99

recombination, 99

reference, 99

reference_material, 99

refine_fraction, 100

refine_k_space, 100

region, 100

Region/Air, 100

Region/barrier, 101

Region/barrier_doped, 101

Region/buffer_doped, 102

Region/bulk, 102

Region/cap, 102

Region/cap_doped, 103

Region/collector, 103

Region/electrolyte, 104

Region/n_side, 104

Region/Nanowire, 105

Region/oxide, 105

Region/p_side, 105

Region/passivation, 106

Region/porous, 106

Region/Well, 106

regions, 107

rel_tolerance, 107

relative_accuracy, 107

relative_tolerance, 107

resultpath, 108

run

 Bulk Si, 12

 Si-Pn Diode, 19

S

save_state, 108

searchpath, 108

Si-Pn Diode

 device, 15

 model, 16

 output, 21

 physic, 19

 run, 19

 solver, 18

simulation

 n-MosFET 2D, 24

simulation_name, 108

solution_method, 109

solve, 109

solver, 109

 Si-Pn Diode, 18

start, 109

statistics, 110

step_tolerance, 110

steps, 110

stop, 110

strain_model_name, 111

strain_simulation, 111

structure, 111

symmetry, 111

T

temperature, 112

TiO₂, 112

type, 112

U

uniform_refinement, 113

V

values, 113

variable, [113](#)

verbose, [113](#)

W

wedge, [114](#)

X

x, [114](#)

x-growth-direction, [114](#)

Y

y-growth-direction, [114](#)

Young, [115](#)

Z

z-growth-direction, [115](#)